OpenClaw Robotic Arm Control

OpenClaw Robotic Arm Control

1. Course Content
   - Course Overview
   - Learning Objectives
2. Prerequisites
3. Robotic Arm Related Control Interfaces
4. Case Demonstration
   - 4.1 Start OpenClaw Interaction
   - 4.2 Get Current Joint Angles of Robotic Arm
   - 4.3 Simultaneously Control 6 Joints of Robotic Arm
   - 4.4 Control Single Robotic Arm Joint
   - 4.5 Restore Robotic Arm to Preset Initial Pose
   - 4.7 Adjust Camera Viewing Angle
5. Core Source Code Analysis
   - 5.1 Six-Joint Simultaneous Control (`ControlArmJointAngles`)
   - 5.2 Single Joint Independent Control (`ControlSingleArmJointAngle`)
   - 5.3 Initial Pose Reset (`InitArmPose`)
   - 5.4 Smart Camera View Adaptive Alignment (`AdjustCameraView`)
   - 5.5 Forward Kinematics and End-Effector Pose Calculation (`GetEndEffectorPose`)

1. Course Content

Course Overview

- OpenClaw robotic arm control is one of the core features of the OpenClaw platform, converting AI natural language instructions into robotic arm control operations through the MCP protocol
- Covers 6 types of robotic arm control interfaces: joint angle acquisition, end-effector pose acquisition, multi-joint simultaneous control, single joint control, camera view adjustment, initial pose recovery

Learning Objectives

1. Understand the interface functions of OpenClaw for controlling robotic arms
2. Master the practical methods and use cases for each robotic arm control interface
3. Be able to optimize grasping and placing accuracy by adjusting configuration file parameters

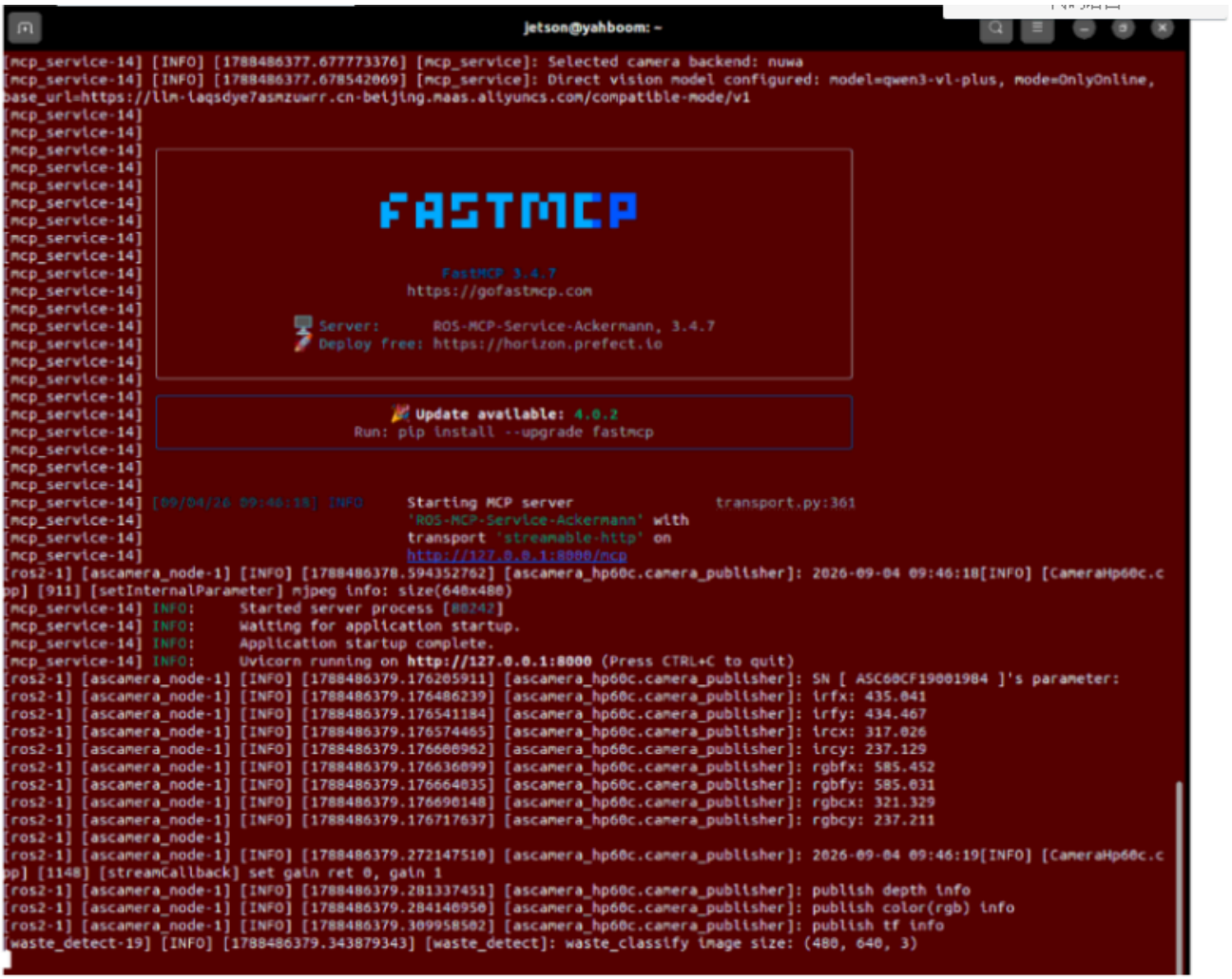
2. Prerequisites

- Start core nodes such as odometry, TF transforms, robotic arm assistance, camera nodes, and MCP service:

```
ros2 launch dofbot_pro_bringup mcp_service.launch.py
```

- If you need voice broadcast responses, use the command with the `launch_openclaw_bridge` parameter, otherwise use the basic command above:

```
ros2 launch dofbot_pro_bringup mcp_service.launch.py   launch_openclaw_bridge:=true
```



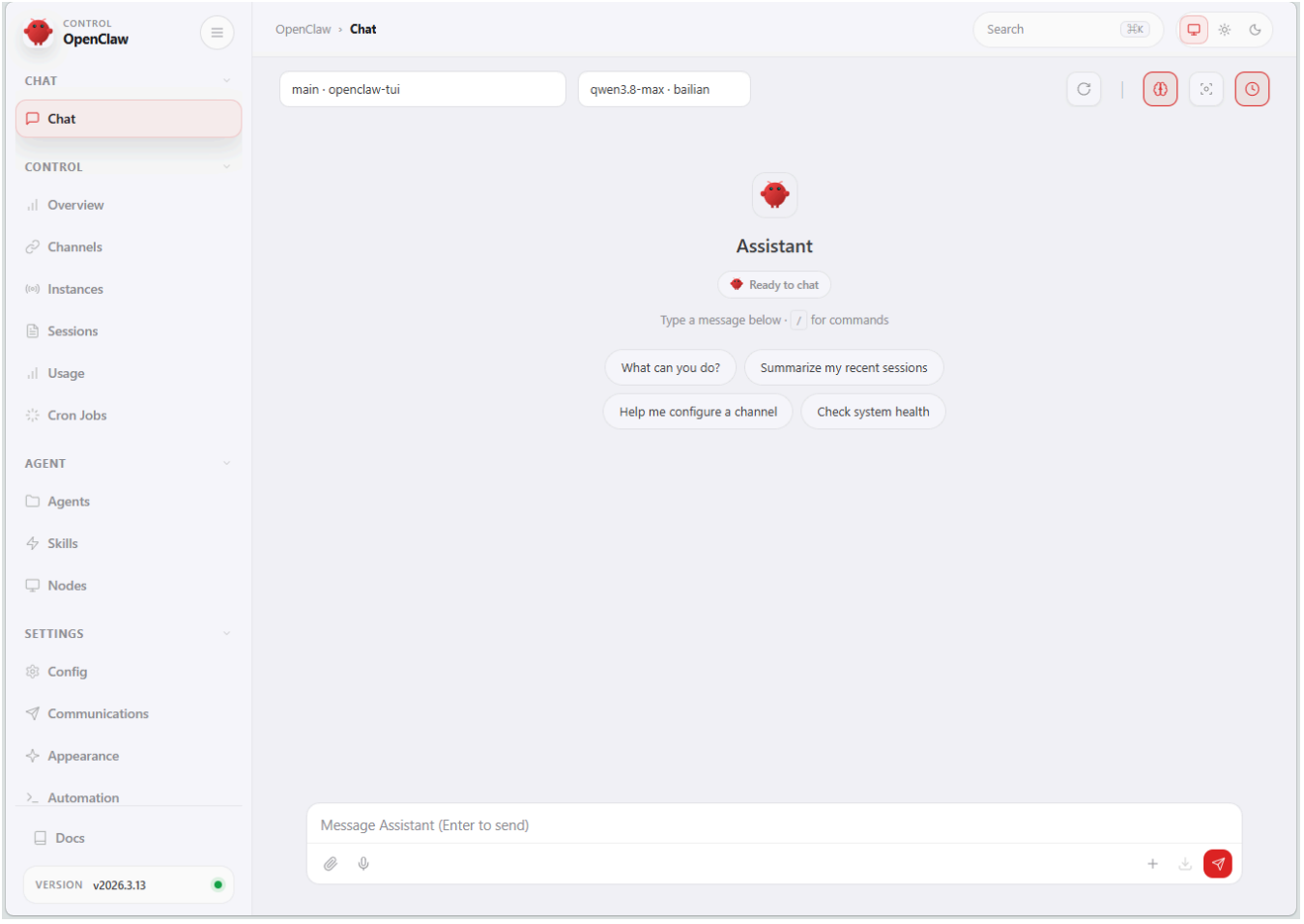
3. Robotic Arm Related Control Interfaces

No.	Tool Name	Function Description
1	GetCurrentArmJointAngles	Get current joint angles of robotic arm
2	ControlSixArmJoint	Simultaneously control 6 joints of robotic arm
3	ControlSingleArmJoint	Control a single robotic arm joint to specified angle
4	InitArmPose	Restore robotic arm to preset initial pose
5	AdjustCameraView	Adjust camera viewing angle
6	GetEndEffectorPose	Get end effector pose

4. Case Demonstration

4.1 Start OpenClaw Interaction

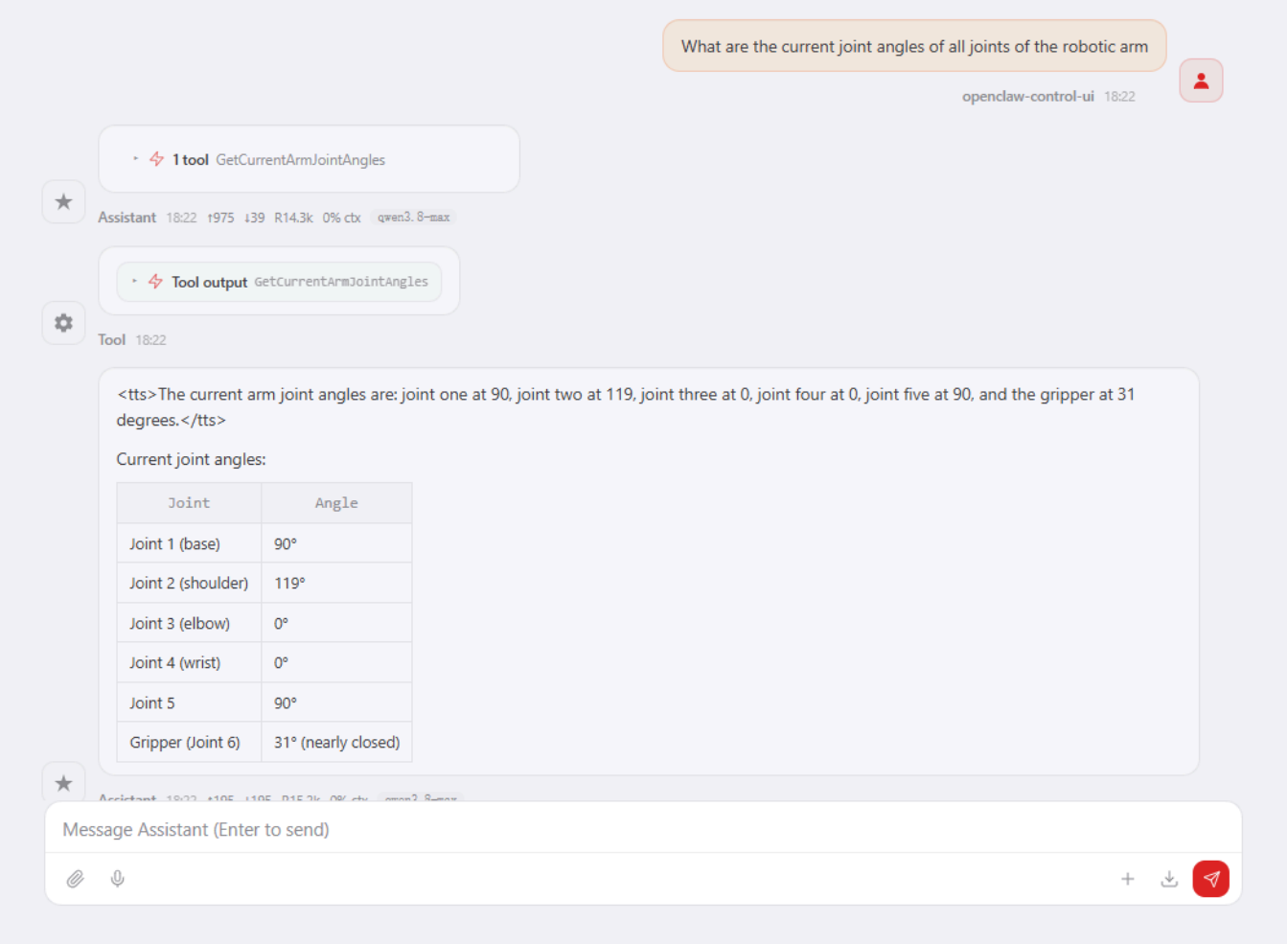
- Use any interaction method to converse with OpenClaw. Here demonstrates using Web Chat



> **Tip**
>
> - Commands can be written according to your own needs. The following are case demonstrations

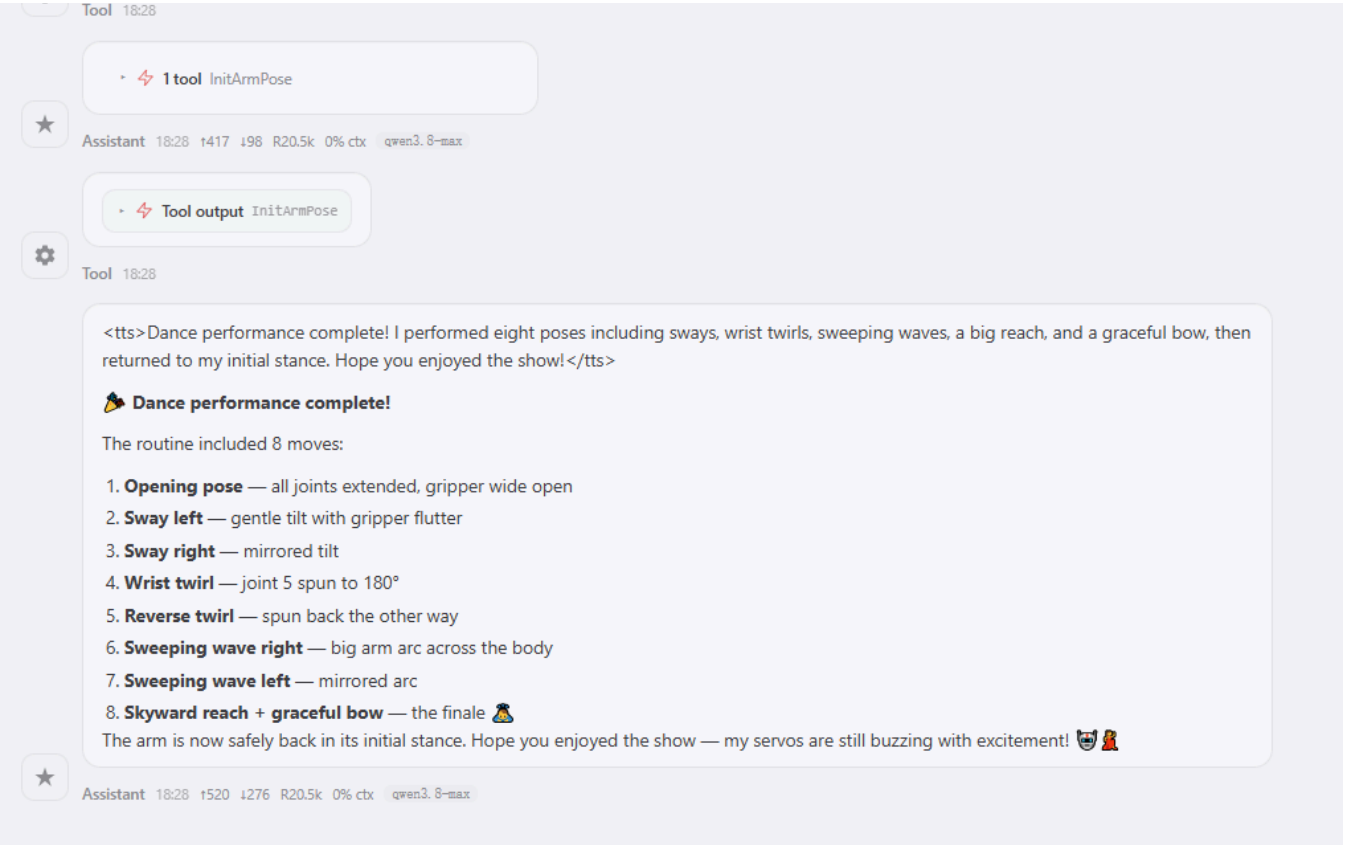
4.2 Get Current Joint Angles of Robotic Arm

- What are the current joint angles of all joints of the robotic arm



4.3 Simultaneously Control 6 Joints of Robotic Arm

- Use the robotic arm to perform a dance for me
- OpenClaw autonomously decides and choreographs robotic arm dance movements. Actual results prevail; different models produce different outputs



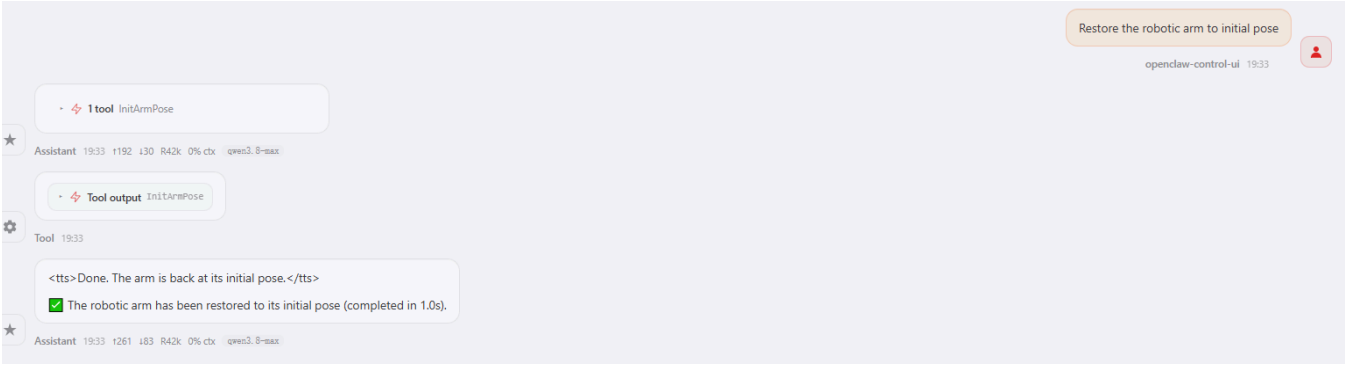
4.4 Control Single Robotic Arm Joint

- Adjust the first joint of the robotic arm to 0 degrees



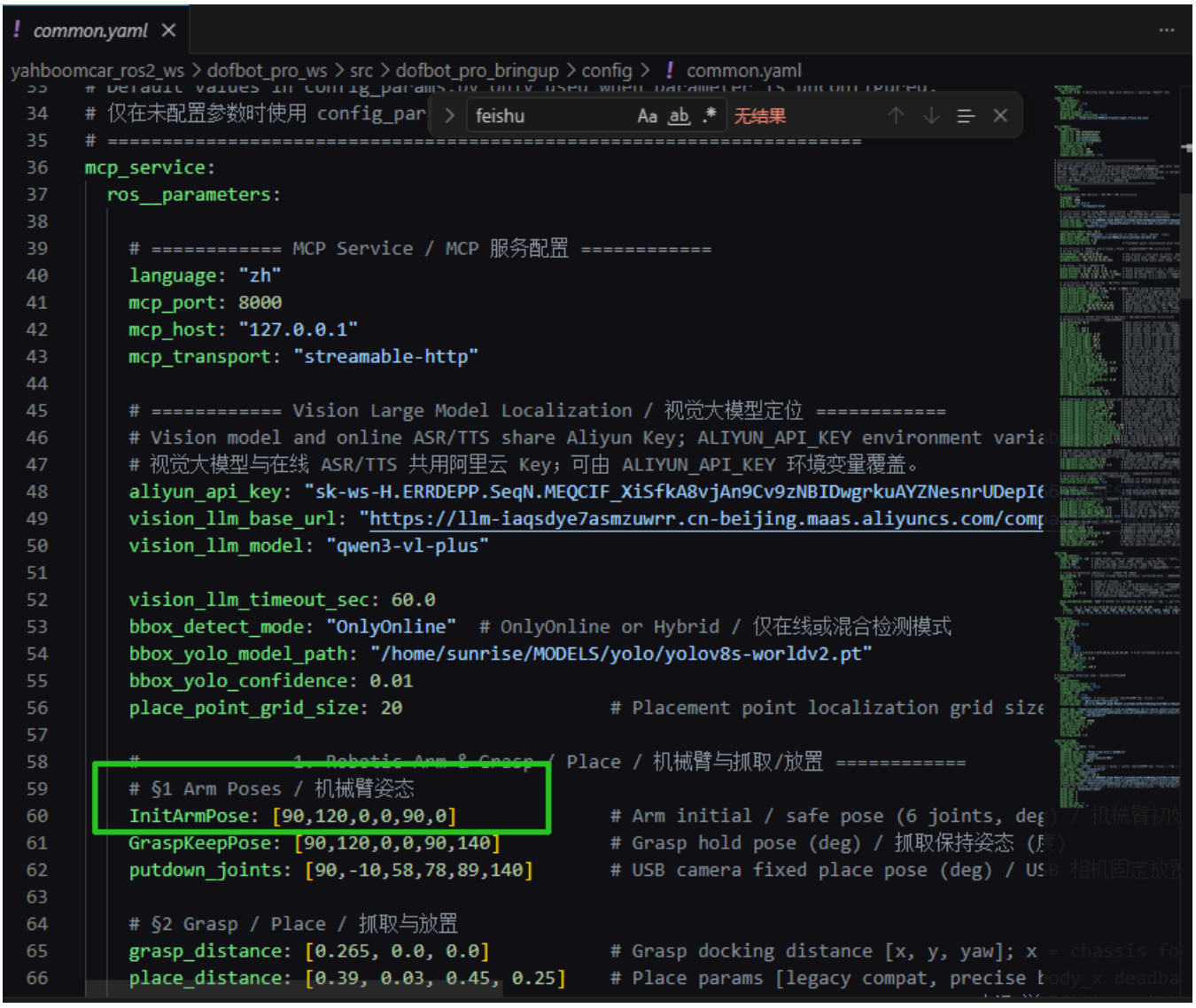
4.5 Restore Robotic Arm to Preset Initial Pose

- Restore the robotic arm to initial pose



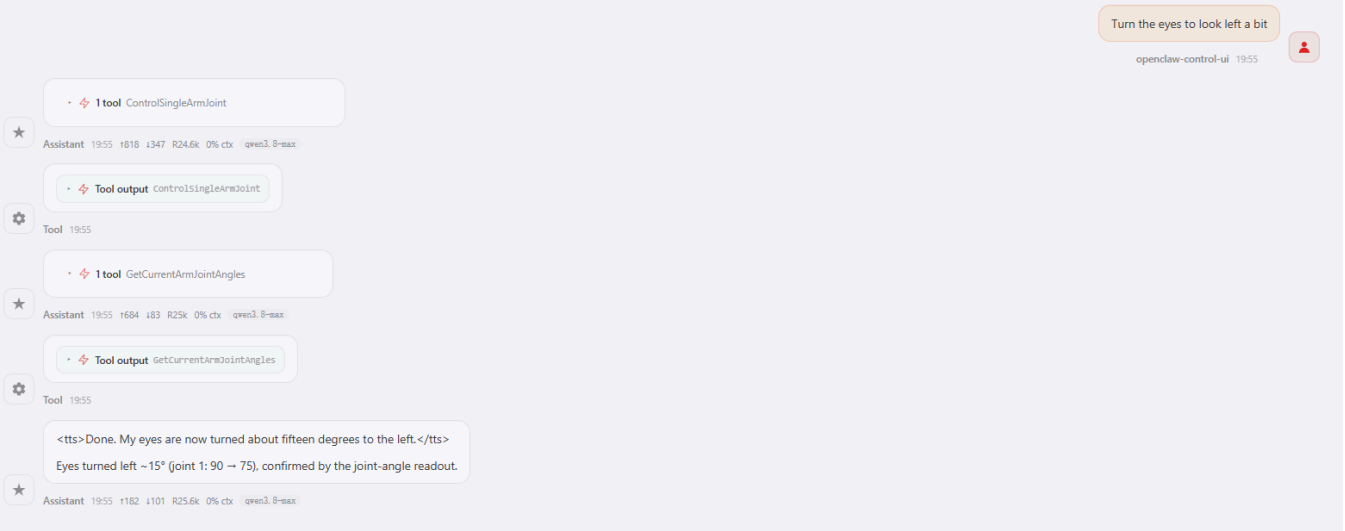
- The initial pose can be modified through the configuration file

```
~/yahboomcar_ros2_ws/dofbot_pro_ws/src/dofbot_pro_bringup/config/common.yaml
```



4.7 Adjust Camera Viewing Angle

- Turn the eyes to look left a bit
- This function is used to conveniently adjust the viewing angle when the robotic arm is working to find targets



5. Core Source Code Analysis

- Core source code path:
  - Controller adapter layer: `~/yahboomcar_ros2_ws/dofbot_pro_ws/src/ackermann_mcp/ackermann_mcp/action_controller.py`
  - Arm limiting and grasp solving: `~/yahboomcar_ros2_ws/dofbot_pro_ws/src/ackermann_mcp/ackermann_mcp/arm/grasp.py`

5.1 Six-Joint Simultaneous Control (`ControlArmJointAngles`)

Code Description:

- `ControlArmJointAngles`: Simultaneously drives all 6 servos to coordinate motion.
- **Physical limit validation (`validate_direct_arm_joints`)**: Strictly validates whether each joint's input value falls within the safe mechanical limit range, preventing servo physical jamming or collision damage:
  - **Joint 1 (Base rotation)**: `[0, 180]`°
  - **Joint 2 (Upper arm pitch)**: `[-60, 180]`°
  - **Joint 3 (Forearm pitch)**: `[-20, 180]`°
  - **Joint 4 (Wrist pitch)**: `[-180, 180]`°
  - **Joint 5 (Wrist rotation)**: `[0, 270]`°
  - **Joint 6 (End effector gripper)**: `[0, 180]`°
- **Gripper angle flip (`invert_joint6=True`)**: The hardware bottom servo 0° represents fully open, 180° represents fully closed. To conform to human intuitive control logic (180=fully open, 0=closed), the function automatically performs `angle = 180 - angle` mirroring.
- **Topic publishing**: Constructs `ArmJoints` message, publishes to `/arm6_joints` topic via `SixJoint_pub`, bottom servo driver smoothly interpolates motion.

```
def ControlArmJointAngles(self, arm_joint_1, arm_joint_2, arm_joint_3,
                          arm_joint_4, arm_joint_5, arm_joint_6, runtime=2000) -> bool:
    """Simultaneously control 6 joints to target angles with runtime"""
    joints = [arm_joint_1, arm_joint_2, arm_joint_3, arm_joint_4, arm_joint_5, arm_joint_6]
    valid, error_msg = validate_direct_arm_joints(joints)
    if not valid:
        self.get_logger().error(error_msg)
        return False
    # invert_joint6=True performs 180 - angle flip, so 180 corresponds to gripper fully open
    self._publish_arm_joints(joints, runtime, invert_joint6=True)
    return True
```

5.2 Single Joint Independent Control (`ControlSingleArmJointAngle`)

Code Description:

- `ControlSingleArmJointAngle`: Independently rotates a specified joint without changing the current pose of the other 5 joints.
- **Parameter validation**: Calls `validate_direct_arm_joint_angle(arm_joint_id, angle)` to individually validate the upper and lower limits of that joint.
- **Gripper compatibility**: If `arm_joint_id == 6`, automatically flips the angle, constructs `ArmJoint` message and publishes to `/arm_joint`.

```
def ControlSingleArmJointAngle(self, arm_joint_id: int, angle: int, runtime: int = 1500) ->
bool:
    """Control a single specified joint of the robotic arm"""
    valid, error_msg = validate_direct_arm_joint_angle(arm_joint_id, angle)
    if not valid:
        self.get_logger().error(error_msg)
        return False

    if arm_joint_id == 6:
        angle = 180 - angle  # Gripper intuitive angle conversion
    self._publish_single_joint(arm_joint_id, angle, runtime)
    return True
```

5.3 Initial Pose Reset (`InitArmPose`)

Code Description:

- `InitArmPose`: Safely folds and stores the robotic arm to a preset safe default folded pose, preventing the arm from extending too far during high-speed cruise driving which could cause center of gravity instability or interfere with surrounding obstacles.
- The pose parameters are dynamically read from the `InitArmPose` key in `common.yaml` (default is `[90, 120, 0, 0, 90, 0]`). After publishing, delay 1.0 seconds to ensure smooth movement completion.

```
def InitArmPose(self) -> bool:
    """Drive the robotic arm back to the safe initial pose specified in config file"""
    self._publish_arm_joints(self.init_arm_pose)
    time.sleep(1.0)
    return True
```

5.4 Smart Camera View Adaptive Alignment (`AdjustCameraView`)

Code Description:

In embodied grasping operations, after the vehicle navigates and docks at a workstation, there often exists physical positioning residual error of several centimeters or degrees. Traditional manual pitch/yaw specification is extremely cumbersome and lacks adaptive capability. `ackermann_mcp` innovatively introduces a **closed-loop camera view alignment algorithm based on navigation high-precision alignment residual feedback**:

1. **Residual acquisition and timeliness validation**: Extracts the last saved `body_x` (longitudinal error), `body_y` (lateral displacement deviation) and `yaw_error` (heading angle deviation) from Nav2 high-precision alignment via `_get_recent_precise_view_errors`, and validates that the data has not expired (not exceeding `precise_error_max_age_sec`).
2. **Mode adaptive decision (`auto_mode`)**:
    - `shelf` **(Shelf workstation mode)**:
        - If longitudinal error `body_x > 0.04m`, the vehicle is docked forward; adjust Joint 2 servo to 135° to look down at the shelf;
        - If `body_x < -0.04m`, the vehicle is docked backward; adjust Joint 2 servo to 115° to look up;
        - Horizontal direction synthesizes `body_y` and `yaw_error` vectors via `_camera_joint1_from_view_errors`, calculates Joint 1 yaw compensation angle (gain coefficient 1.2), and limits compensation angle within `±20°` protection range;
    - `general` **(General mode)**:
        - Keeps pitch unchanged, focuses on fine-tuning Joint 1 based on lateral and heading errors, centering the target in the field of view;
    - `None`: Keeps current joints stationary, only calls `SeeWhat()` to refresh and return the latest image.

```
def _camera_joint1_from_view_errors(self, body_y: float, yaw_error: float) -> tuple[int, str]:
    """Calculate Joint 1 target steering angle based on navigation residuals"""
    yaw_deg = math.degrees(yaw_error)
    y_gain = 1.2
    yaw_offset = max(-20.0, min(20.0, yaw_deg / 9.0 * 20.0))
    y_offset = max(-20.0, min(20.0, body_y / 0.10 * 20.0 * y_gain))
    same_direction = y_offset == 0.0 or yaw_offset == 0.0 or (y_offset > 0.0) == (yaw_offset > 0.0)
    raw_offset = y_offset + yaw_offset if same_direction else y_offset
    total_offset = max(-20.0, min(20.0, raw_offset))
```

```
    joint1 = int(round(90.0 - total_offset))
    return joint1, f"body_y={body_y:.3f}m, yaw_error={yaw_deg:.1f}deg -> joint1={joint1}°"

def AdjustCameraView(self, auto_mode: str = None) -> list:
    """Adaptive camera view adjustment"""
    if auto_mode not in (None, "shelf", "general"):
        return [False, "Invalid auto_mode; expected 'shelf', 'general', or None"]

    if auto_mode in ("shelf", "general"):
        errors = self._get_recent_precise_view_errors(label="AdjustCameraView")
        if errors is None:
            return [False, "No recent successful navigation precision result for auto
adjustment"]

        body_x, body_y, yaw_error = errors
        if auto_mode == "shelf":
            if body_x > 0.04:
                self._publish_single_joint(2, 135, 500)   # Vehicle front, look down
            elif body_x < -0.04:
                self._publish_single_joint(2, 115, 500)   # Vehicle back, look up

        # Horizontal yaw compensation
        joint1, reason = self._camera_joint1_from_view_errors(body_y, yaw_error)
        self._publish_single_joint(1, joint1, 500)
        self.get_logger().info(f"AdjustCameraView: {reason}")
        time.sleep(0.6)

    self._reset_joints()
    return self.SeeWhat()  # Automatically capture and return latest image after adjustment
```

5.5 Forward Kinematics and End-Effector Pose Calculation (`GetEndEffectorPose`)

Code Description:

- `GetEndEffectorPose()`: Calculates the current spatial Cartesian coordinates `(x, y, z)` and Euler angle posture `(roll, pitch, yaw)` of the robotic arm end-effector gripper through Forward Kinematics (FK).
- First calls `_get_current_arm_joints()` to read the angles of the first 5 spatial drive joints, constructs `ArmKinemarics.Request(kin_name="fk")`;
- Through the ROS 2 service client `self.arm_kin_client`, requests the underlying kinematics solver node, and returns the calculated end-effector spatial pose rounded to 2 decimal places, providing data basis for upper-level advanced spatial path planning and collision detection.

```
def GetEndEffectorPose(self) -> list:
    """Calculate current end-effector pose through FK forward kinematics"""
    current_joints = self._get_current_arm_joints()
    if len(current_joints) < 5:
```

```
        self.get_logger().warn("Failed to get current joint angles, cannot execute FK
calculation")
        return []

    response_pose: ArmKinemarics.Response = self.arm_kin_client.call(
        self._build_fk_request_from_joints(current_joints))

    return [
        round(response_pose.x, 2), round(response_pose.y, 2), round(response_pose.z, 2),
        round(response_pose.roll, 2), round(response_pose.pitch, 2),
round(response_pose.yaw, 2)
    ]
```